PROJECT REPORT

Tic-Tac-Toe Game with Computer Opponent Using Python

1. Introduction

Tic-Tac-Toe is a game that is played on a 3×3 grid with two players that take turns placing their symbols in empty boxes. One player places X while the other one places O. The main objective of the game is to be the first player to have three of one's symbols in a row, horizontally, vertically, or diagonally.

In this project, a Tic-Tac-Toe game was designed using Python to enable a human player to play against the computer. The Minimax algorithm was used to make the computer opponent intelligent. Through this project, I have learned how to design a Python game as well as how artificial intelligence (AI) can be used to improve performance of the computer.

This project was useful in helping me to practice and improve my Python coding skills as well as to gain deeper insight into how computer programs work.

2. Objectives

This project had the following objectives:

1. Design and develop a Tic-Tac-Toe game in Python.

2. Enable the game to allow a human player to play against the computer.

3. Learn how to implement the Minimax algorithm.

4. Display the game board after each turn.

5. Determine when the human player, computer, or draw wins the game.

6. Practise and apply Python concepts such as functions, classes, loops, conditions, and lists.

3. Tools and Technologies

The following tools and technologies were used to develop this application:

- Programming Language: Python

- IDE: PyCharm

- Algorithm: Minimax algorithm

- Interface: Terminal

4. Project Description

In this game, the player uses X while the computer uses O to mark the game board. Upon running the program, the game board will be displayed with numbers indicating the positions of the game board. The player will select a position by entering the number corresponding to the position.

The computer will then determine the best possible move using the Minimax algorithm to select the best position and place the computer's symbol O.

The game will continue playing until either player wins by having three symbols in a row, column, or diagonal or until all the positions on the game board have been filled, resulting in a Draw.

This game was designed to run on a terminal, and thus, no additional tools were required.

5. Implementation and Working

The computer program was divided into functions to enable easier handling and comprehension of the code. Several functions were defined, including displaying the game board, making human moves, making computer moves, determining the winner, and determining if the game was a draw.

The Minimax algorithm was also implemented to enable the computer to make the best moves possible.

The functions were implemented as separate entities, and thus it was easy to detect and correct errors and even add new features.

6. Algorithm

The game works as follows:

1. The game board will be created and displayed.

2. The human player will be asked to select a position to place their symbol X.

3. The validity of the position selected will be determined.

4. The human player's symbol will be placed in the selected position.

5. It will be determined whether the human player has won or the game is drawn.

6. If the game is not over, the computer will use the Minimax algorithm to select the best move and place the computer's symbol O.

7. The computer's move will be displayed.

8. It will be determined whether the computer has won or the game is drawn.

9. Steps 3 to 8 will be repeated until the game ends.

10. The results of the game will be displayed.

7. Testing and Results

This project was tested using test cases to determine whether the program behaves as expected.

Various test cases were designed and are discussed below.

Test Case 1: Human player selects a valid position and makes a valid move

Expected Results: The game board should be updated to reflect the new position of the X.

Test Case 2: Human player selects an invalid position

Expected Results: An error message should be displayed indicating that the selected position is not valid.

Test Case 3: Human player selects an already occupied position

Expected Results: A warning message should be displayed, and the human player should be asked to retry.

Test Case 4: Human player wins the game

Expected Results: A message indicating that the human player has won the game should be displayed.

Test Case 5: Computer wins the game

Expected Results: A message indicating that the computer has won the game should be displayed.

Test Case 6: The game is a draw

Expected Results: A message indicating that the game is a draw should be displayed.

Through these test cases, the game's performance, including error detection and handling mechanism, was evaluated.

The Minimax algorithm was also effective in making the computer opponent hard to beat.

8. Challenges Faced

Some of the challenges faced during the development of this project include:

1. Understanding how the Minimax algorithm works.

2. Handling various erroneous inputs from the human player.

3. Ensuring that the game ends as soon as one of the players wins.

4. Dividing the program into smaller functions that perform distinct tasks.

The first challenge was quite challenging because the Minimax algorithm is not intuitive.

However, by implementing the algorithm step by step, a comprehensive understanding of how it works was achieved.

The second challenge was also quite challenging because the game required extensive error checking to identify and handle erroneous inputs.

The third challenge was addressed by determining whether the game was over right after a move was made.

The last challenge was quite easy to overcome since the program was broken down into manageable units.

These challenges played a crucial role in enhancing the learning process of this project.

9. Learning Outcomes

This project was successful in enabling me to learn several concepts related to Python programming.

The knowledge gained in this project includes:

1. How to apply Python to develop functional computer programs such as games.

2. How to implement Minimax algorithm to make computer opponents smart.

3. How to properly design computer programs by dividing them into manageable functions.

4. How to make effective use of loops, conditions, lists, and user input in Python.

This project also helped me to enhance my existing knowledge about Python, including improving my coding skills.

The practical experience of developing a computer game enabled me to better understand the workings of various Python concepts.

10. Conclusion

To conclude, the project was a great learning experience. Through it, I managed to learn how to develop a computer game using Python that enables a human player to play against a computer.

I was also able to learn various concepts related to Python programming, including loops, conditions, lists, and user input. Despite encountering several challenges while developing this program, I was able to overcome them and gain a deeper understanding of how Python works.

This project also helped me to develop insight into how artificial intelligence can be applied in computer programs to improve their performance.